

Guía Rápida de Instrucciones MIPS Assembler

Este documento es una referencia rápida de las instrucciones más utilizadas en la arquitectura MIPS. Úsala como material de apoyo para entender y modificar el código del laboratorio.

1. Instrucciones Aritméticas y Lógicas

Estas instrucciones realizan cálculos sobre los registros.

Instrucción	Ejemplo	Explicación
add	add \$t0, \$t1, \$t2	Suma: $t0 = t1 + t2$ (con control de overflow).
addu	addu \$t0, \$t1, \$t2	Suma sin signo: $t0 = t1 + t2$.
sub	sub \$t0, \$t1, \$t2	Resta: $t0 = t1 - t2$.
addi	addi \$t0, \$t1, 5	Suma inmediata: $t0 = t1 + 5$.
mul	mul \$t0, \$t1, \$t2	Multiplicación: $t0 = t1 * t2$.
and	and \$t0, \$t1, \$t2	AND lógico bit a bit.
or	or \$t0, \$t1, \$t2	OR lógico bit a bit.
sll	sll \$t0, \$t1, 2	Shift Left Logical: Desplaza $t1$ a la izquierda 2 bits (equivale a multiplicar por 4).

2. Instrucciones de Transferencia de Datos

MIPS es una arquitectura **Load/Store**, lo que significa que solo estas instrucciones pueden acceder a la memoria RAM.

```
# Ejemplo de carga de una palabra (word)
lw $t0, 0($s0)    # Carga en $t0 el contenido de la memoria en la dirección $s0 + 0

# Ejemplo de almacenamiento de una palabra
sw $t1, 4($s1)    # Guarda el valor de $t1 en la memoria en la dirección $s1 + 4
```

- **la (Load Address):** Carga la dirección de una etiqueta (variable) en un registro.
 - `la $s0, vector_x # $s0` ahora apunta al inicio de `vector_x`.
- **li (Load Immediate):** Carga un valor constante directamente en un registro.
 - `li $t0, 10 # $t0 = 10`.

3. Instrucciones de Control de Flujo

Permiten realizar saltos y bucles.

Instrucción	Ejemplo	Explicación
<code>beq</code>	<code>beq \$t0, \$t1, etiqueta</code>	Salta a <code>etiqueta</code> si <code>\$t0 == \$t1</code> .
<code>bne</code>	<code>bne \$t0, \$t1, etiqueta</code>	Salta a <code>etiqueta</code> si <code>\$t0 != \$t1</code> .
<code>j</code>	<code>j etiqueta</code>	Salto incondicional a <code>etiqueta</code> .
<code>jal</code>	<code>jal funcion</code>	Jump and Link: Salta a una función y guarda la dirección de retorno en <code>\$ra</code> .
<code>jr</code>	<code>jr \$ra</code>	Jump Register: Salta a la dirección contenida en <code>\$ra</code> (usado para volver de funciones).

4. Pseudoinstrucciones Comunes

Son instrucciones que el ensamblador traduce a una o más instrucciones reales para facilitar la programación.

- **move \$t0, \$t1:** Copia el valor de `$t1` en `$t0`. (Se traduce como `addu $t0, $t1, $zero`).
- **clear \$t0:** Pone el registro `$t0` a cero. (Se traduce como `add $t0, $zero, $zero`).

5. Ejemplo Comentado (Estructura de un Bucle)

```
.data
    array: .word 10, 20, 30    # Definir un array de 3 enteros
    size:  .word 3             # Tamaño del array

.text
main:
    la $s0, array              # Cargar dirección base del array
    lw $s1, size                # Cargar el tamaño (3)
    li $t0, 0                  # Inicializar índice i = 0

loop:
    beq $t0, $s1, fin          # Si i == size, salir del bucle

    # --- Cuerpo del bucle ---
    sll $t1, $t0, 2            # t1 = i * 4 (calcular offset en bytes)
```

```
    addu $t2, $s0, $t1 # t2 = dirección de array[i]
    lw $t3, 0($t2)     # t3 = valor de array[i]

    # (Aquí se harían operaciones con el valor t3)

    addi $t0, $t0, 1    # Incrementar índice i++
    j loop              # Volver al inicio del bucle

fin:
    li $v0, 10          # Código para terminar programa
    syscall
```